

Instead of using equals which compares all the text in the email to match with 'gmail', we use the contains clause which means gmail can be present anywhere in the email field.

Sorting

We can easily sort the list of emails which we retrieved in the previous step by calling the sort method on the RealmResults object. Use the sort method which accepts the name of the field on the basis of which items are sorted in the ascending order by default.

Refer to SNIPPET 12 from HERE <http://dgit.in/RealmCode>

Aggregating values

Realm also lets you aggregate results and obtain the minimum, maximum, average or sum of all values for a field. In our case, if the Person model class had a field called age, we could perform the following operations.

Refer to SNIPPET 13 from HERE <http://dgit.in/RealmCode>

Iterating

There are several ways of iterating through the RealmResults obtained after performing a query. Don't forget to use a transaction if you plan on updating any or all of the Person objects.

Refer to SNIPPET 14 from HERE <http://dgit.in/RealmCode>

Synchronous & Asynchronous Queries

The queries we executed so far run on the UI thread. Given that Realm loads data lazily and Realm uses a highly efficient structure to store and retrieve data, synchronous queries perform pretty fast compared to SQLite. To run a query asynchronously, we first create the query and use the findAllAsync method instead of the regular findAll method.

Refer to SNIPPET 15 from HERE <http://dgit.in/RealmCode>

The query is not blocking and immediately returns a RealmResults<Drop>. This is a promise similar to the concept of Future in standard Java. The query will continue to run in

a background thread, and once it completes it will update the returned instance of RealmResults. If you want to be notified when the query completes and the RealmResults object is updated, you can register a RealmChangeListener. This listener will be called every time the RealmResults are updated to reflect the latest changes in the Realm. You can only use asynchronous queries on a Looper thread.

Refer to SNIPPET 16 from HERE <http://dgit.in/RealmCode>

In the final step, simply register and unregister for the listener inside your Activity or Fragment to prevent memory leaks.

Refer to SNIPPET 17 from HERE <http://dgit.in/RealmCode>

Advanced Concepts

Several advanced concepts in Realm dealing with its configuration, encryption, notifications, json processing and multithreading are covered in detail here. (<https://realm.io/docs/java/latest/#realms>)

Conclusion

Realm is a free product as of March 16, 2016 and has been in use since 2012 in commercial apps for both Android and IOS. It has a growing community with 24x7 support available on its GitHub repo for the most common difficulties you will encounter. It accelerates development time compared to SQLite while being truly cross-platform in nature.

Links

Java documentation for Realm <https://realm.io/docs/java/latest/>
If you want to learn more on how to use realm in a practical android application, check the complete bucket drops series which covers it <https://goo.gl/WvX8GD> >

The author has published a certificate course "Launch Your First Android App in Just 16 Hours!" on the Pyoopil App. Pyoopil is a personal learning network, which allows you to take amazing courses and keeps you connected to your mentors and peers. Use the code 'DIGIT' to get a 70% discount on this certificate course. Here is the link: <http://dgit.in/PyoopilApp>

*Nodes of Interest



*Tax Benefit for Devs

>>A pending bill in the senate of New York aims to provide tax credit to open source developers

<http://dgit.in/OpnSrcTaxCredit>



*MAME Open Source

>>After announcing it last year, Multiple Arcade Machine Emulator is now open source.

<http://dgit.in/MAMEFree>



*Netflix's coding

>> Netflix tech blog on how code is built at Netflix, the global content streaming giant.

<http://dgit.in/NetflixCoding>